



# *CE 222 – Computer Organization and Assembly Language (COAL)*

## **Lecture 4**

Dr. Asad Mahmood

Jan. 26, 2026

# *Lecture Outline*

- Announcements (Assignment and Quiz 1)
- Outline of Previous Lecture:
  - **Chapter 1 - Computer Abstractions and Technology**
    - 1.4 Under the Covers
    - 1.5 Technologies for Building Processors and Memory
    - 1.6 Performance
- Outline to Today's Lecture:
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

# Chapter 1

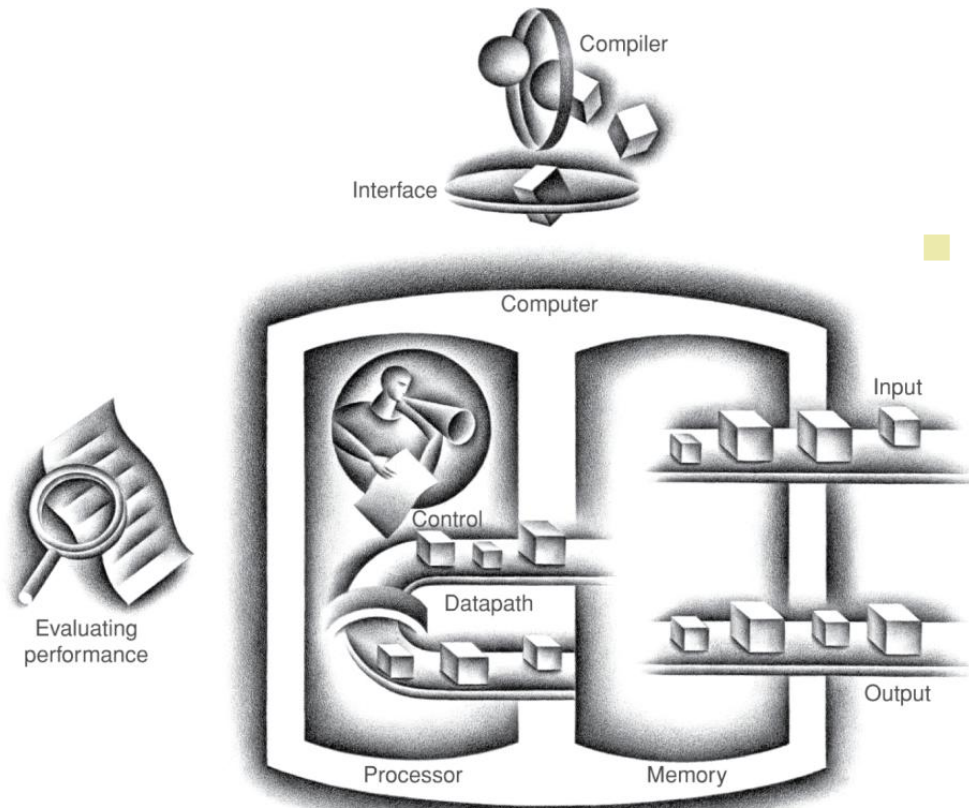
## Computer Abstractions and Technology

# Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - [1.4 Under the Covers](#)
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

# Components of a Computer

## The BIG Picture



- Same components for all kinds of computer
  - Desktop, server, embedded
- **Input/output** includes
  - **User-interface devices**
    - Display, keyboard, mouse
  - **Storage devices**
    - Hard disk, CD/DVD, flash
  - **Network adapters**
    - For communicating with other computers

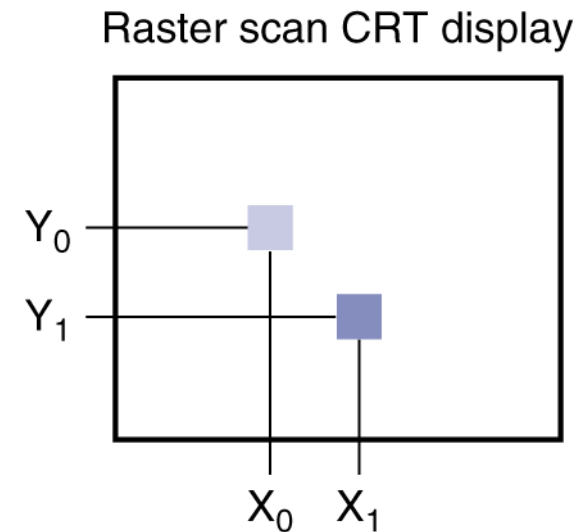
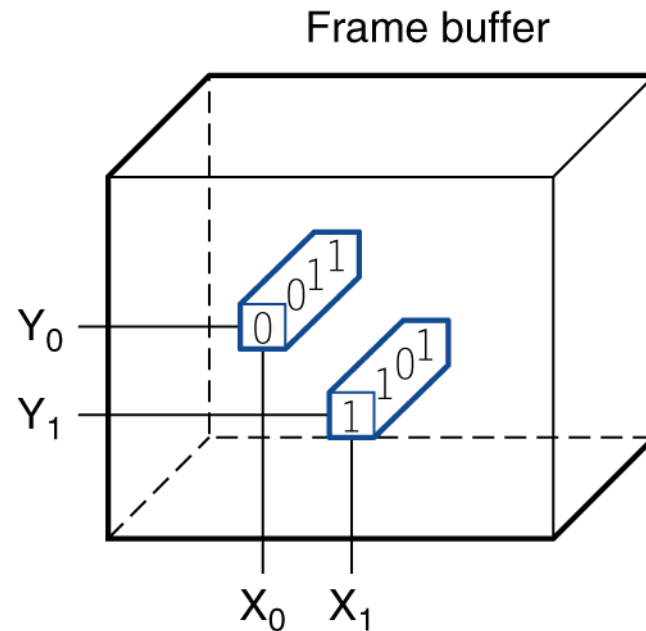
# Touchscreen

- PostPC device
- Supersedes keyboard and mouse
- Resistive and Capacitive types
  - Most tablets, smart phones use capacitive
  - Capacitive allows multiple touches simultaneously

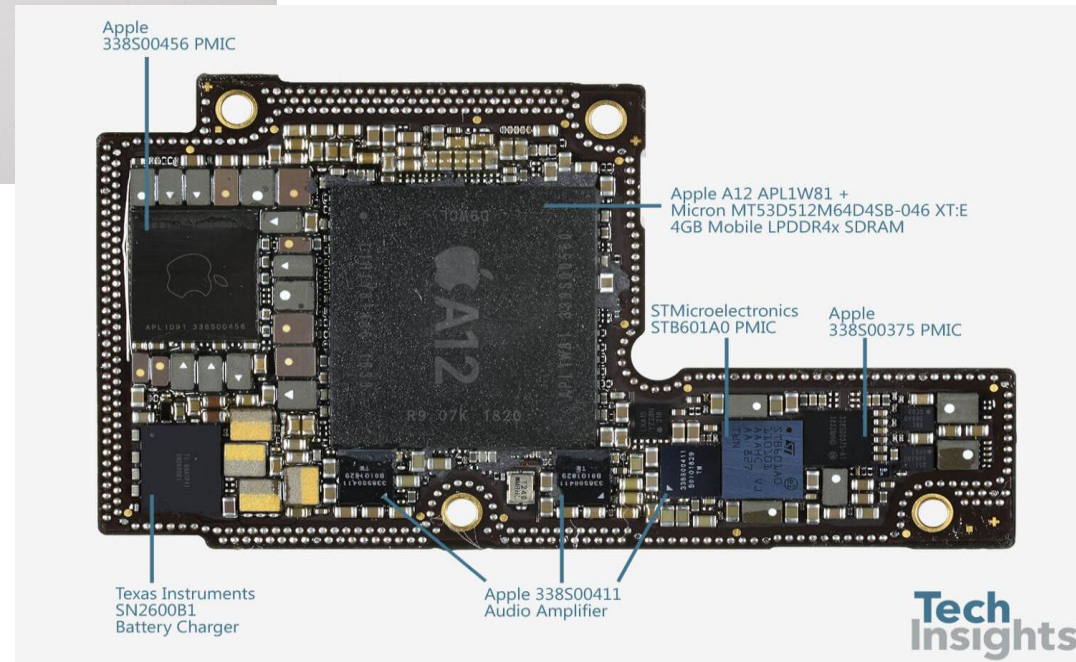
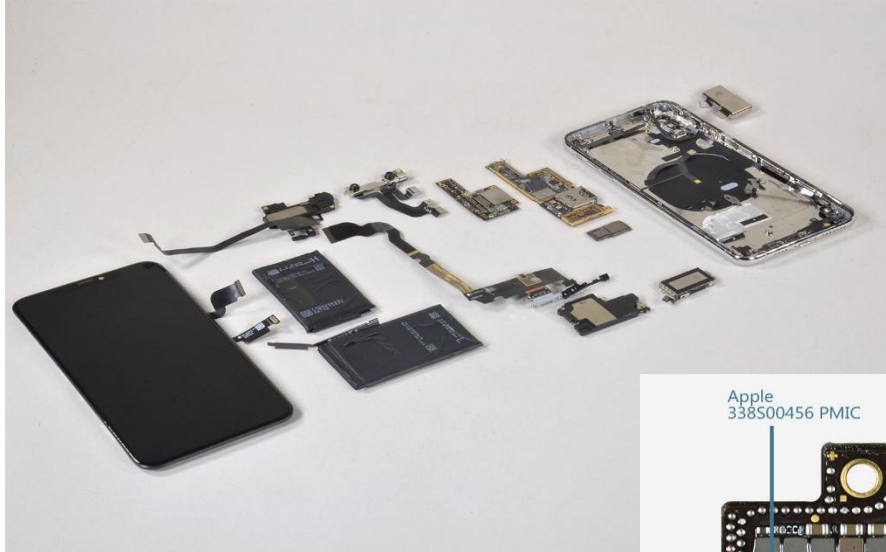


# Through the Looking Glass

- LCD screen: picture elements (pixels)
  - Mirrors content of frame buffer memory



# Opening the Box



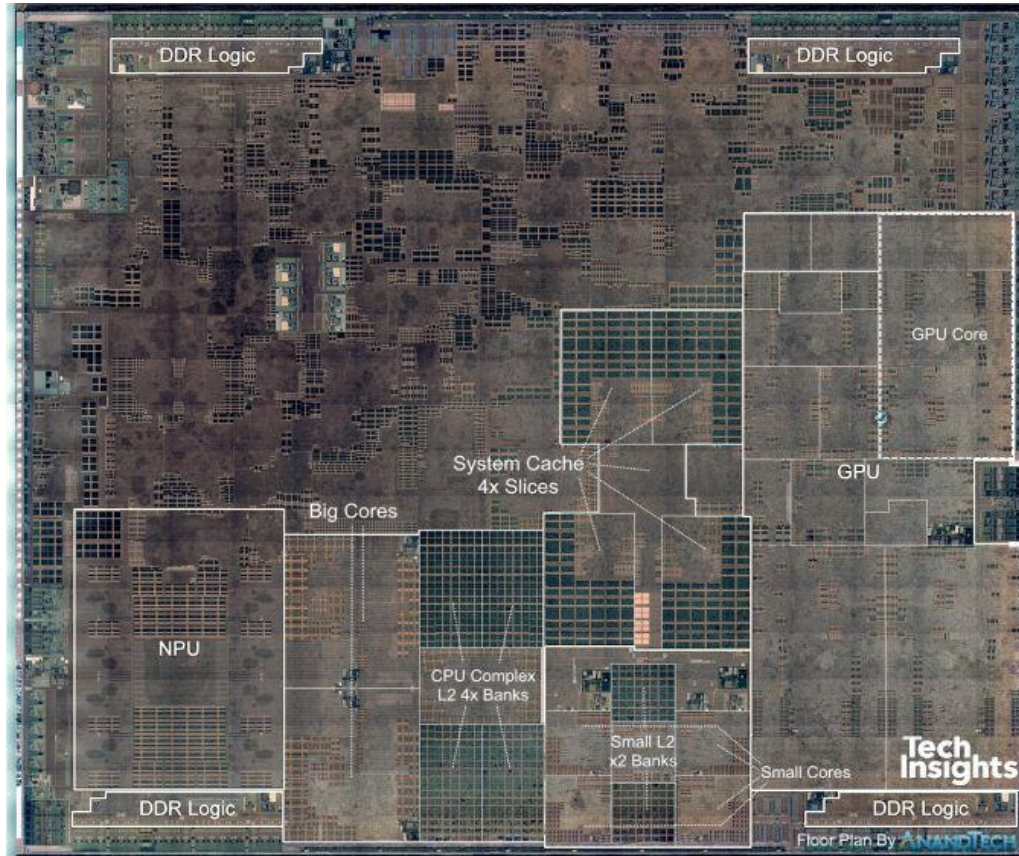
Tech  
Insights

# Inside the Processor (CPU)

- **Datapath:** performs operations on data
- **Control:** sequences datapath, memory, ...
- **Cache memory**
  - Small fast SRAM memory for immediate access to data

# Inside the Processor

- A12 processor



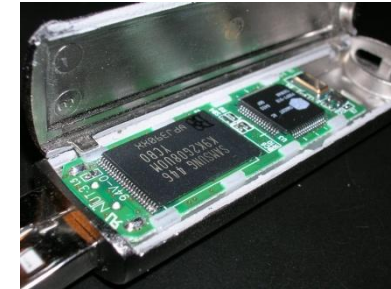
# Abstractions

## The BIG Picture

- Abstraction helps us deal with complexity
  - Hide lower-level detail
- Instruction set architecture (ISA)
  - The hardware/software interface
- Application binary interface
  - The ISA plus system software interface
- Implementation
  - The details underlying and interface

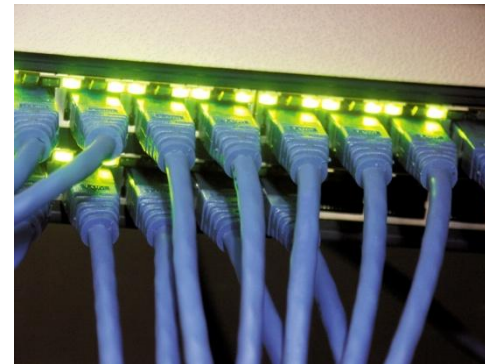
# A Safe Place for Data

- Volatile main memory
  - Loses instructions and data when power off
- Non-volatile secondary memory
  - Magnetic disk
  - Flash memory
  - Optical disk (CDROM, DVD)



# Networks

- Communication, resource sharing, nonlocal access
- Local area network (LAN): Ethernet
- Wide area network (WAN): the Internet
- Wireless network: WiFi, Bluetooth

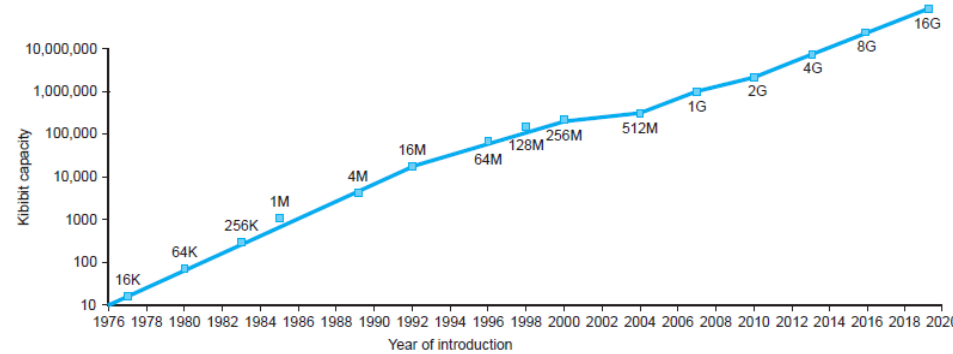


# Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - [1.5 Technologies for Building Processors and Memory](#)
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

# Technology Trends

- Electronics technology continues to evolve
  - Increased capacity and performance
  - Reduced cost
  - **Moore's Law**



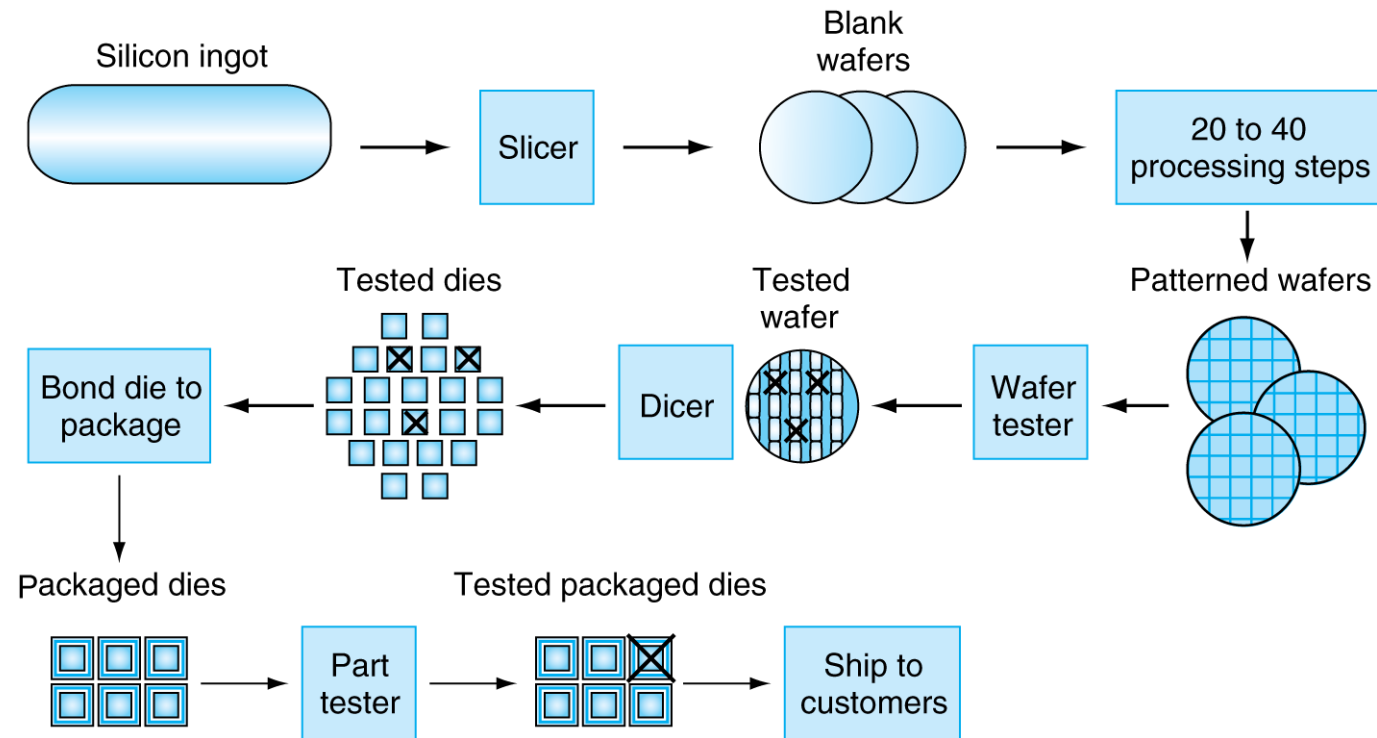
DRAM capacity

| Year | Technology                 | Relative performance/cost |
|------|----------------------------|---------------------------|
| 1951 | Vacuum tube                | 1                         |
| 1965 | Transistor                 | 35                        |
| 1975 | Integrated circuit (IC)    | 900                       |
| 1995 | Very large scale IC (VLSI) | 2,400,000                 |
| 2013 | Ultra large scale IC       | 250,000,000,000           |

# Semiconductor Technology

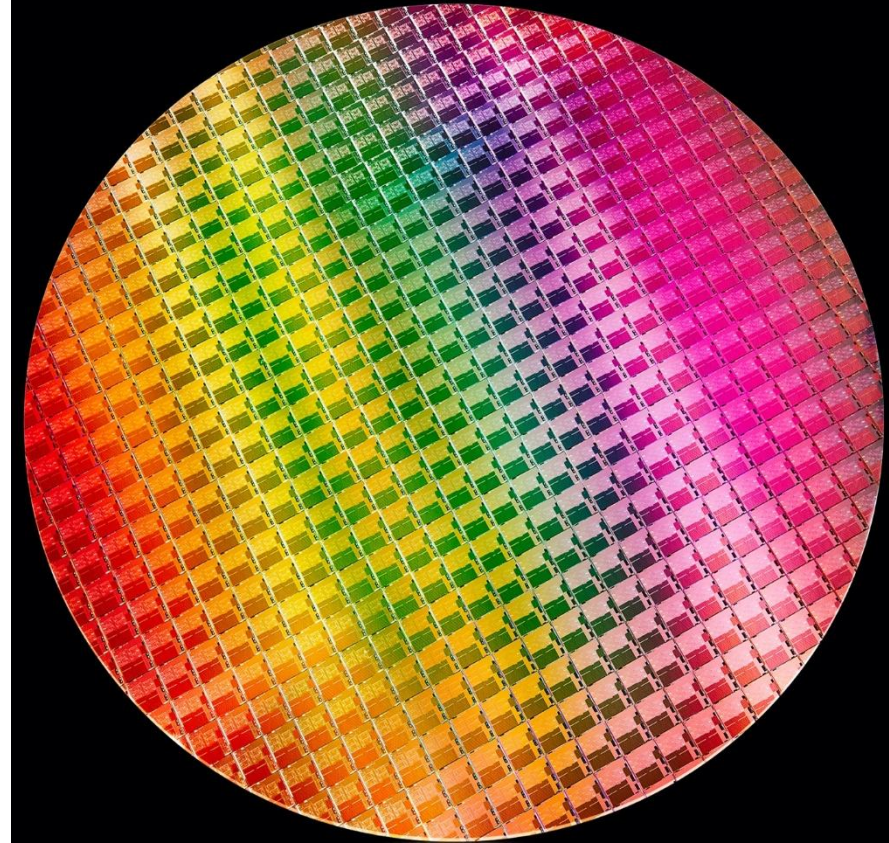
- Silicon: **semiconductor**
- Add materials to transform properties:
  - Conductors
  - Insulators
  - Switch

# Manufacturing ICs



- Yield: proportion of working dies per wafer

# Intel® Core 10<sup>th</sup> Gen



- 300mm wafer, 506 chips, 10nm technology
- Each chip is 11.4 x 10.7 mm

# Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

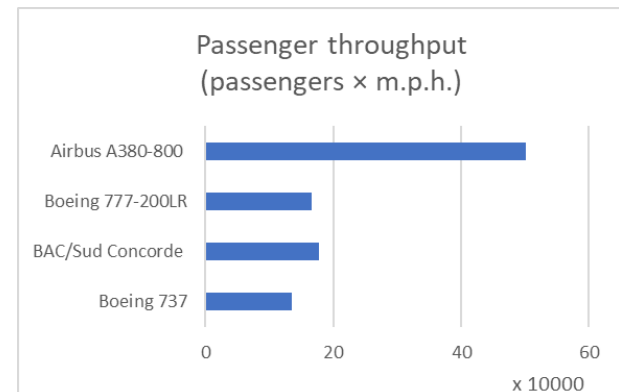
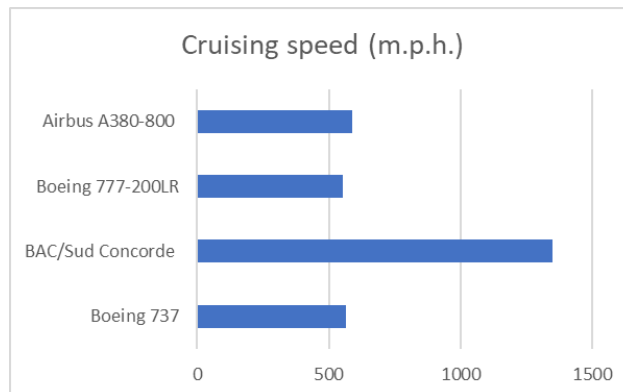
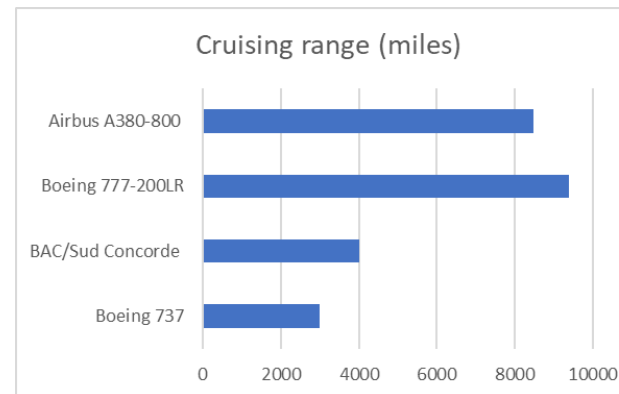
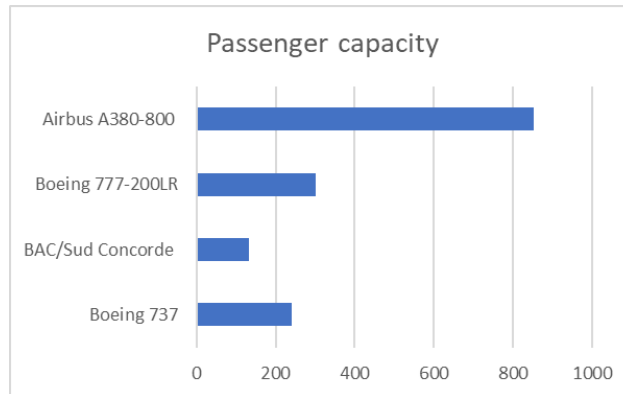
- Nonlinear relation to area and defect rate
  - Wafer cost and area are fixed
  - Defect rate determined by manufacturing process
  - Die area determined by architecture and circuit design

# Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - **1.6 Performance**
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

# Defining Performance

- Very important to define computer performance but not easy!
- Which airplane has the best performance?



# Response Time and Throughput

- We can define computer performance in several distinct ways
- Response time
  - How long it takes to do a task
  - Focus of an individual user
- Throughput
  - Total work done per unit time e.g., Total tasks / transactions/... per hour
  - Focus of a data-center/server
- How are response time and throughput affected by
  1. Case 1 - Replacing the processor with a faster version?
  2. Case 2 - Adding more processors?
- We'll focus on response time for now...

# Relative Performance

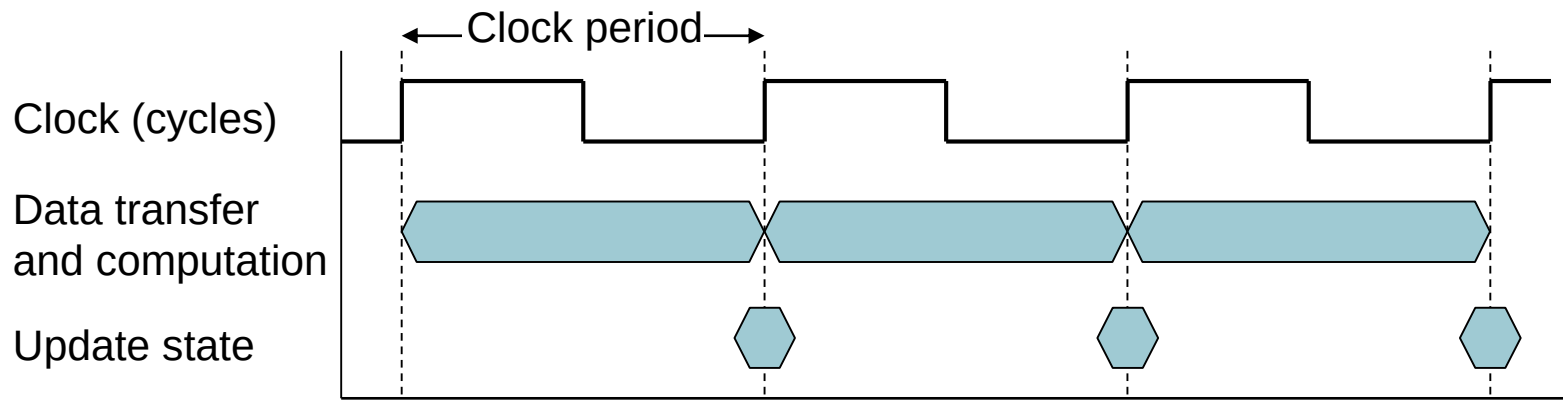
- Let us define the *Performance* of a computer in terms of its Execution Time
- What it means when one says “X is  $n$  time faster than Y”?
- **Example:** If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

# Measuring Execution Time

- Execution time (Seconds/Program) can be measured in a number of ways, e.g.
  1. Elapsed time = Total response time, including all aspects i.e. Processing, I/O, OS overhead, idle time
  2. CPU time = Time spent processing a given job without including I/O time, other jobs' shares
    - Comprises user CPU time and system CPU time
- Which one of the above two is experienced by the user?
- We will focus on CPU performance in this chapter

# CPU Clocking

- Operation of digital hardware governed by a **constant-rate clock**, which **can be used to define CPU time**



- Clock period (s) and clock frequency / rate (cycles/s)
- Relating Clock period/Freq. with CPU time
- How performance can be improved?
- Often a tradeoff involved!

# CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
  - Aim for 6s CPU time
  - Can do faster clock, but causes  $1.2 \times$  clock cycles
- How fast must Computer B clock be?

# Instruction Count and CPI

- Instruction Count (# of Instructions) for a program
  - Determined by program, ISA and compiler
- Relating instruction count with the CPU execution time
- Average cycles per instruction (CPI) affected by instruction mix

| Components of performance          | Units of measure                               |
|------------------------------------|------------------------------------------------|
| CPU execution time for a program   | Seconds for the program                        |
| Instruction count                  | Instructions executed for the program          |
| Clock cycles per instruction (CPI) | Average number of clock cycles per instruction |
| Clock cycle time                   | Seconds per clock cycle                        |

**FIGURE 1.15** The basic components of performance and how each is measured.

# CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
  - Computer B: Cycle Time = 500ps, CPI = 1.2
  - Same ISA
  - Which is faster, and by how much?
- 
- CPI in more detail

# CPI Example

- For a high-level language statement, a compiler writer is considering two code sequences for a computer that require the following instruction counts for 3 classes of instructions A, B and C

| Class            | A | B | C |
|------------------|---|---|---|
| CPI for class    | 1 | 2 | 3 |
| IC in sequence 1 | 2 | 1 | 2 |
| IC in sequence 2 | 4 | 1 | 1 |

- Which code sequence executes the most instructions?
- What is the CPI for each sequence?
- Which will be faster?

# Performance Summary

## The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Performance depends on
  - **Algorithm**: affects IC, possibly CPI
  - **Programming language**: affects IC, CPI
  - **Compiler**: affects IC, CPI
  - **Instruction set architecture**: affects IC, CPI,  $T_c$

# Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - [1.7 The Power Wall](#)
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading